02157 Functional Programming
Week 3:
Structuring and Typing

Sebastian Mödersheim
Based on slides by Michael Reichhardt Hansen

September 18, 2026

Outline

1. Review
2. On functions as first-class citizens
3. Type Inference

Exercises

- HR 1.5: the Fibonacci numbers

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \text{ for } n > 1 \end{aligned}$$

- HR 2.2: $\texttt{pow}(\texttt{s}, \texttt{n}) = \underbrace{s \cdot s \cdot \ldots \cdot s}_{n}$ for string s and int n.
- HR 4.7: `multiplicity x xs` – compute the number of times `x` occurs in `xs`.
- Polynome exercise from slide **??** ($\approx$ HR 4.22).
- Merge sort: idea explained in class on blackbord
  Define helper functions (they are all in the List library, but easy to write yourself):
  - ★ `length: 'a list->int` length of a list
    e.g. `length [1;2;3] = 3`
  - ★ `split: int->'a list->('a list,'a list)` split a list into two after a given number of elements
    e.g. `split 2 [1;2;3;4;5] = ([1;2],[3;4;5])`
  - ★ `merge: ('a list,'a list) -> 'a list` merge a pair of sorted lists into one sorted list.

  Put it together into mergesort.

Outline

1. Review
2. On functions as first-class citizens
3. Type Inference

Functions as "first-class citizens"

- functions can be passed as arguments to functions
- functions can be returned as values of functions

Functions as "first-class citizens"

- functions can be passed as arguments to functions
- functions can be returned as values of functions

like any other kind of value.

Functions as "first-class citizens"

- functions can be passed as arguments to functions
- functions can be returned as values of functions

like any other kind of value.

There is nothing special about functions in functional languages

Functions as "first-class citizens"

- functions can be passed as arguments to functions
- functions can be returned as values of functions

like any other kind of value.

There is nothing special about functions in functional languages

A function that takes a function as argument or produces a function as result is also called a higher-order function.

Functions as "first-class citizens"

- functions can be passed as arguments to functions
- functions can be returned as values of functions

like any other kind of value.

There is nothing special about functions in functional languages

A function that takes a function as argument or produces a function as result is also called a higher-order function.

Higher-order functions are useful

- succinct code
- highly parameterized programs
- Program libraries typically contain many such functions

An example

Suppose that we have a cube with side length s, containing a liquid with density ρ. The weight of the liquid is then given by $\rho \cdot s^3$:

```
let weight ro s = ro * s ** 3.0;;
val weight : float -> float -> float
```

An example

Suppose that we have a cube with side length s, containing a liquid with density ρ. The weight of the liquid is then given by $\rho \cdot s^3$:

```
let weight ro s = ro * s ** 3.0;;
val weight : float -> float -> float
```

We can make *partial evaluations* to define functions for computing the weight of a cube of either water or methanol:

```
let waterWeight = weight 1000.0;;
val waterWeight : (float -> float)

waterWeight 2.0;;
val it : float = 8000.0
```

An example

Suppose that we have a cube with side length s, containing a liquid with density ρ. The weight of the liquid is then given by $\rho \cdot s^3$:

```
let weight ro s = ro * s ** 3.0;;
val weight : float -> float -> float
```

We can make *partial evaluations* to define functions for computing the weight of a cube of either water or methanol:

```
let waterWeight = weight 1000.0;;
val waterWeight : (float -> float)

waterWeight 2.0;;
val it : float = 8000.0

let methanolWeight = weight 786.5 ;;
val methanolWeight : (float -> float)

methanolWeight 2.0;;
val it : float = 6292.0
```

An example

Suppose that we have a cube with side length s, containing a liquid with density ρ. The weight of the liquid is then given by $\rho \cdot s^3$:

```
let weight ro s = ro * s ** 3.0;;
val weight : float -> float -> float
```

We can make *partial evaluations* to define functions for computing the weight of a cube of either water or methanol:

```
let waterWeight = weight 1000.0;;
val waterWeight : (float -> float)

waterWeight 2.0;;
val it : float = 8000.0

let methanolWeight = weight 786.5 ;;
val methanolWeight : (float -> float)

methanolWeight 2.0;;
val it : float = 6292.0
```

The formula $\rho \cdot s^3$ is represented just once in the program

Currying and Uncurrying

The process of turning a function on pairs (tuples) into a higher-order function is called currying. The opposite process is called uncurrying.

Currying and Uncurrying

The process of turning a function on pairs (tuples) into a higher-order function is called currying. The opposite process is called uncurrying.

Consider declarations:

```
let wC ro s = ro * s ** 3.0;;
val wC : float -> float -> float

let wUC(ro, s) = ro * s ** 3.0;;
val wUC : ro:float * s:float -> float
```

- `wC` is the curried version of `wUC`
- `wUC` is the uncurried version of `wC`

Currying and Uncurrying

The process of turning a function on pairs (tuples) into a higher-order function is called currying. The opposite process is called uncurrying.

Consider declarations:

```
let wC ro s = ro * s ** 3.0;;
val wC : float -> float -> float

let wUC(ro, s) = ro * s ** 3.0;;
val wUC : ro:float * s:float -> float
```

- `wC` is the curried version of `wUC`
- `wUC` is the uncurried version of `wC`

Have a look at exercise HR 2.13:

- declare functions for curring and uncurring.

```
curry:   ('a * 'b -> 'c) -> 'a -> 'b -> 'c
uncurry: ('a -> 'b -> 'c) -> 'a * 'b -> 'c
```

A well-known example: function composition

Function composition: $(f \circ g)(x) = f(g(x))$

For example, if $f(y) = y + 3$ and $g(x) = x^2$, then $(f \circ g)(z) = z^2 + 3$.

A well-known example: function composition

Function composition: $(f \circ g)(x) = f(g(x))$

For example, if $f(y) = y + 3$ and $g(x) = x^2$, then $(f \circ g)(z) = z^2 + 3$.

The infix operator << in F# denotes function composition:

```
let f y = y+3;;

let g x = x*x;;

let h = f << g;;          // h = (f o g)
val h : int -> int

h 4;;                     // h(4) = (f o g)(4)
val it : int = 19
```

- An infix operator appears between the arguments

Infix functions

The prefix version $(\oplus)$ of an infix operator $\oplus$ is a curried function, that is, higher-order function where argument are supplied one by one

For example:

```
(<<);;
val it : (('a -> 'b) -> ('c -> 'a) -> 'c -> 'b)
```

Infix functions

The prefix version $(\oplus)$ of an infix operator $\oplus$ is a curried function, that is, higher-order function where argument are supplied one by one

For example:

```
(<<);;
val it : (('a -> 'b) -> ('c -> 'a) -> 'c -> 'b)
```

- The argument is a function `'a -> 'b`

Infix functions

The prefix version $(\oplus)$ of an infix operator $\oplus$ is a curried function, that is, higher-order function where argument are supplied one by one

For example:

```
(<<);;
val it : (('a -> 'b) -> ('c -> 'a) -> 'c -> 'b)
```

- The argument is a function `'a -> 'b`
- The value is a function `('c -> 'a) -> 'c -> 'b`

Infix functions

The prefix version $(\oplus)$ of an infix operator $\oplus$ is a curried function, that is, higher-order function where argument are supplied one by one

For example:

```
(<<);;
val it : (('a -> 'b) -> ('c -> 'a) -> 'c -> 'b)
```

- The argument is a function `'a -> 'b`
- The value is a function `('c -> 'a) -> 'c -> 'b`

Declaration

<< is a built-in function

```
let (<<) f g x = f(g x);;
```

Infix functions

The prefix version $(\oplus)$ of an infix operator $\oplus$ is a curried function, that is, higher-order function where argument are supplied one by one

For example:

```
(<<);;
val it : (('a -> 'b) -> ('c -> 'a) -> 'c -> 'b)
```

- The argument is a function `'a -> 'b`
- The value is a function `('c -> 'a) -> 'c -> 'b`

Declaration

<< is a built-in function

```
let (<<) f g x = f(g x);;
```

Infix operators are written as strings of special characters including

```
!  %  &  *  +  -  /  <  =  >  ?  @  ^  \  ~
```

Consult F# specification for complete rules.

The built-in infix function @

List.append is a higher-order function from the `List` library:

- `List.append` $[x_0; \ldots; x_{n-1}]\,[y_0; \ldots; x_{m-1}] =$
  $[x_0; \ldots; x_{n-1}; y_0; \ldots; x_{m-1}]$

The built-in infix function @

List.append is a higher-order function from the `List` library:

- `List.append` $[x_0; \ldots; x_{n-1}]\,[y_0; \ldots; x_{m-1}] =$
  $[x_0; \ldots; x_{n-1}; y_0; \ldots; x_{m-1}]$

There is a convenient infix notation for `List.append` *xs ys* in F#:

xs `@` *ys*

Function expressions

Function Expressions

Let x be a variable of type τ_0 and e be an expression of type τ. Then also

$$\texttt{fun}\ x\ \texttt{->}\ e$$

is an expression that defines a function that takes an x and returns e. Thus, the resulting expression is of type $\tau_0 \to \tau$.

- We will later discuss lambda calculus, the logical basis for functional programming.
- The `fun x -> e` in F# corresponds to $\lambda x.e$ in the lambda calculus and reads: "the function of x given by e".

Anonymous function with pattern matching

We often would like to have pattern matching on variables, i.e.,

```
fun x ->
  match x with
  | p1 -> e1
  ...
  | pn -> en
```

Anonymous function with pattern matching

We often would like to have pattern matching on variables, i.e.,

```
fun x ->
  match x with
  | p1 -> e1
  ...
  | pn -> en
```

F# offers the following syntactic sugar for that:

```
function
 | p1 -> e1
 ...
 | pn -> en
```

Example

An anonymous function computing the number of days in a month:

```
function
| 2  -> 28   // February
| 4  -> 30   // April
| 6  -> 30   // June
| 9  -> 30   // September
| 11 -> 30   // November
| _  -> 31;; // All other months
 val it : int -> int = <fun:clo@17-2>
```

Example

An anonymous function computing the number of days in a month:

```
function
| 2  -> 28   // February
| 4  -> 30   // April
| 6  -> 30   // June
| 9  -> 30   // September
| 11 -> 30   // November
| _  -> 31;; // All other months
 val it : int -> int = <fun:clo@17-2>

it 2;;
val it : int = 28
```

Anonymous functions: Example 3

Function expressions with general patterns, e.g.

```
function
| 2          -> 28  // February
|4|6|9|11    -> 30  // April, June, September, November
| _          -> 31  // All other months
;;
```

Anonymous functions: Example 3

Function expressions with general patterns, e.g.

```
function
| 2          -> 28  // February
|4|6|9|11    -> 30  // April, June, September, November
| _          -> 31  // All other months
;;
```

Exploits an or pattern in the second clause

Simple functions expressions with currying

The expression

$$\texttt{fun}\ x\ y\ \cdots\ z\ \rightarrow e$$

has the same meaning as

$$\texttt{fun}\ x \rightarrow (\texttt{fun}\ y \rightarrow (\cdots\ (\texttt{fun}\ z \rightarrow e)\cdots))$$

Simple functions expressions with currying

The expression

$$\texttt{fun}\ x\ y\ \cdots\ z\ \rightarrow e$$

has the same meaning as

$$\texttt{fun}\ x \rightarrow (\texttt{fun}\ y \rightarrow (\cdots\ (\texttt{fun}\ z \rightarrow e)\cdots))$$

It denotes a function with type

$$t_x \rightarrow (t_y \rightarrow (\cdots(t_z \rightarrow t_e)\cdots)) \text{ where } x : t_x, y : t_y, z : t_z \text{ and } e : t_e$$

Simple functions expressions with currying

The expression

$$\texttt{fun}\ x\ y\ \cdots\ z\ \rightarrow e$$

has the same meaning as

$$\texttt{fun}\ x \rightarrow (\texttt{fun}\ y \rightarrow (\cdots\ (\texttt{fun}\ z \rightarrow e)\cdots))$$

It denotes a function with type

$$t_x \rightarrow (t_y \rightarrow (\cdots(t_z \rightarrow t_e)\cdots)) \text{ where } x : t_x, y : t_y, z : t_z \text{ and } e : t_e$$

For example: The function below takes an integer as argument and returns a function of type `int -> int` as value:

```
fun x y -> x + x*y;;
val it : int ->  int -> int = <fun:clo@2-1>
```

Simple functions expressions with currying

The expression

$$\texttt{fun}\ x\ y\ \cdots\ z\ \rightarrow e$$

has the same meaning as

$$\texttt{fun}\ x \rightarrow (\texttt{fun}\ y \rightarrow (\cdots\ (\texttt{fun}\ z \rightarrow e)\cdots))$$

It denotes a function with type

$$t_x \rightarrow (t_y \rightarrow (\cdots (t_z \rightarrow t_e)\cdots)) \text{ where } x : t_x, y : t_y, z : t_z \text{ and } e : t_e$$

For example: The function below takes an integer as argument and returns a function of type `int -> int` as value:

```
fun x y -> x + x*y;;
val it : int ->  int -> int = <fun:clo@2-1>

let f = it 2;;
val f : ( int -> int)
```

Simple functions expressions with currying

The expression

$$\texttt{fun}\ x\ y\ \cdots\ z\ \rightarrow e$$

has the same meaning as

$$\texttt{fun}\ x \rightarrow (\texttt{fun}\ y \rightarrow (\cdots\ (\texttt{fun}\ z \rightarrow e)\cdots))$$

It denotes a function with type

$$t_x \rightarrow (t_y \rightarrow (\cdots(t_z \rightarrow t_e)\cdots)) \text{ where } x : t_x, y : t_y, z : t_z \text{ and } e : t_e$$

For example: The function below takes an integer as argument and returns a function of type `int -> int` as value:

```
fun x y -> x + x*y;;
val it : int ->  int -> int = <fun:clo@2-1>

let f = it 2;;
val f : ( int -> int)

f 3;;
val it : int = 8
```

Outline

1. Review
2. On functions as first-class citizens
3. **Type Inference**

Types and type checking

Purposes:

- Modelling, readability: types are used to indicate the intention behind a program

Types and type checking

Purposes:

- Modelling, readability: types are used to indicate the intention behind a program

- Safety, efficiency: "Well-typed programs do not go wrong"

Robin Milner

Types and type checking

Purposes:

- Modelling, readability: types are used to indicate the intention behind a program

- Safety, efficiency: "Well-typed programs do not go wrong"

  Robin Milner

  ★ Actually, a well-typed program can still have bugs, but a good type system will rule out a lot of buggy programs.

  ⇒ It is a very useful restriction to require well-typed programs

Types and type checking

Purposes:

- Modelling, readability: types are used to indicate the intention behind a program

- Safety, efficiency: "Well-typed programs do not go wrong"

  Robin Milner

  - ★ Actually, a well-typed program can still have bugs, but a good type system will rule out a lot of buggy programs.
    - ⇒ It is a very useful restriction to require well-typed programs
  - ★ Some ill-typed programs may still be fine, but typically it is not difficult to re-write them into well-typed ones.
    - ⇒ Restriction to well-typed is in practice not burden.

Types and type checking

Purposes:

- Modelling, readability: types are used to indicate the intention behind a program

- Safety, efficiency: "Well-typed programs do not go wrong"

  Robin Milner

  - ★ Actually, a well-typed program can still have bugs, but a good type system will rule out a lot of buggy programs.
    - ⇒ It is a very useful restriction to require well-typed programs
  - ★ Some ill-typed programs may still be fine, but typically it is not difficult to re-write them into well-typed ones.
    - ⇒ Restriction to well-typed is in practice not burden.

- Catch errors at compile time

  - ★ Checks of types are not needed at runtime

Types and type checking

Purposes:

- Modelling, readability: types are used to indicate the intention behind a program

- Safety, efficiency: "Well-typed programs do not go wrong"

  Robin Milner

  - ★ Actually, a well-typed program can still have bugs, but a good type system will rule out a lot of buggy programs.
    - ⇒ It is a very useful restriction to require well-typed programs
  - ★ Some ill-typed programs may still be fine, but typically it is not difficult to re-write them into well-typed ones.
    - ⇒ Restriction to well-typed is in practice not burden.

- Catch errors at compile time

  - ★ Checks of types are not needed at runtime

A type checker is an algorithm used at an early phase in the compiler to reject programs containing type errors.

Type inference is an algorithm to automatically calculate types of expressions without use of explicit type annotations.

Type inference

The type system of F# allows for polymorphic types, that is, types with many forms. Polymorphic types are expressed using type variables 'a, 'b, 'c, ...
Textbooks (and Sebastian) often write greek letters for type variables like $\alpha, \beta, \gamma, \ldots$.

Type inference

The type system of F# allows for polymorphic types, that is, types with many forms. Polymorphic types are expressed using type variables 'a, 'b, 'c, ...
Textbooks (and Sebastian) often write greek letters for type variables like $\alpha, \beta, \gamma, \ldots$.

The *most general type* or *principal type* is inferred by the system.

Examples:

```
let id x = x
val id : 'a -> 'a

let pair x y = (x,y)
val pair : 'a -> 'b -> 'a * 'b
```

Type inference

The type system of F# allows for polymorphic types, that is, types with many forms. Polymorphic types are expressed using type variables 'a, 'b, 'c, ...
Textbooks (and Sebastian) often write greek letters for type variables like $\alpha, \beta, \gamma, \ldots$.

The *most general type* or *principal type* is inferred by the system.

Examples:

```
let id x = x
val id : 'a -> 'a

let pair x y = (x,y)
val pair : 'a -> 'b -> 'a * 'b
```

The inferred types are most general in the sense that all other types for id and pair are instances of the inferred types.

Type inference

The type system of F# allows for polymorphic types, that is, types with many forms. Polymorphic types are expressed using type variables 'a, 'b, 'c, ...
Textbooks (and Sebastian) often write greek letters for type variables like $\alpha, \beta, \gamma, \ldots$.

The *most general type* or *principal type* is inferred by the system.

Examples:

```
let id x = x
val id : 'a -> 'a

let pair x y = (x,y)
val pair : 'a -> 'b -> 'a * 'b
```

The inferred types are most general in the sense that all other types for id and pair are instances of the inferred types.

By the type of a function, we (usually) mean the most general type

Type inference

The type system of F# allows for polymorphic types, that is, types with many forms. Polymorphic types are expressed using type variables 'a, 'b, 'c, ...
Textbooks (and Sebastian) often write greek letters for type variables like $\alpha, \beta, \gamma, \ldots$.

The *most general type* or *principal type* is inferred by the system.

Examples:

```
let id x = x
val id : 'a -> 'a

let pair x y = (x,y)
val pair : 'a -> 'b -> 'a * 'b
```

The inferred types are most general in the sense that all other types for id and pair are instances of the inferred types.

By the type of a function, we (usually) mean the most general type

Remark: identity function id is a built-in function

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$
  where τ_5 represents whatever will be the type of the `match...` expression.

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$ where τ_5 represents whatever will be the type of the `match...` expression.
- Now we add type constraints:
  - ★ The result type τ_5 must be the same type as the result `ys` in case C 1: $\tau_5 = \tau_2$

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$ where τ_5 represents whatever will be the type of the `match...` expression.
- Now we add type constraints:
  - ★ The result type τ_5 must be the same type as the result `ys` in case C 1: $\tau_5 = \tau_2$
  - ★ τ_5 must also be the same type as the result in case C 2: $\texttt{x::(xtail @ ys)} : \tau_5$

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$ where τ_5 represents whatever will be the type of the `match...` expression.
- Now we add type constraints:
  - ★ The result type τ_5 must be the same type as the result `ys` in case C 1: $\tau_5 = \tau_2$
  - ★ τ_5 must also be the same type as the result in case C 2: $\texttt{x::(xtail @ ys)} : \tau_5$
  - ★ For builtin `::` we have $\texttt{(::)} : \alpha \to \alpha\ \texttt{list} \to \alpha\ \texttt{list}$ where α can be any type.
    - ▶ We need to instantiate $\alpha = \tau_3$ here since $x : \tau_3$.

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$ where τ_5 represents whatever will be the type of the `match...` expression.
- Now we add type constraints:
  - ★ The result type τ_5 must be the same type as the result `ys` in case C 1: $\tau_5 = \tau_2$
  - ★ τ_5 must also be the same type as the result in case C 2: $\texttt{x::(xtail @ ys)} : \tau_5$
  - ★ For builtin `::` we have $\texttt{(::)} : \alpha \to \alpha\ \texttt{list} \to \alpha\ \texttt{list}$ where α can be any type.
    - ▶ We need to instantiate $\alpha = \tau_3$ here since $x : \tau_3$.
  - ★ Thus $\tau_5 = \tau_3\ \texttt{list}$ and $\texttt{xtail @ ys} : \tau_3\ \texttt{list}$
  - ★ Thus $\texttt{xtail} : \tau_1$ because `xtail` is used as first argument of `@`.

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$ where τ_5 represents whatever will be the type of the `match...` expression.
- Now we add type constraints:
  - ★ The result type τ_5 must be the same type as the result `ys` in case C 1: $\tau_5 = \tau_2$
  - ★ τ_5 must also be the same type as the result in case C 2: $\texttt{x::(xtail @ ys)} : \tau_5$
  - ★ For builtin `::` we have $\texttt{(::)} : \alpha \to \alpha\ \texttt{list} \to \alpha\ \texttt{list}$ where α can be any type.
    - ▶ We need to instantiate $\alpha = \tau_3$ here since $x : \tau_3$.
  - ★ Thus $\tau_5 = \tau_3\ \texttt{list}$ and $\texttt{xtail @ ys} : \tau_3\ \texttt{list}$
  - ★ Thus $\texttt{xtail} : \tau_1$ because `xtail` is used as first argument of `@`.
  - ★ For the pattern `x::xtail` to be type correct, we must thus hold: $\tau_1 = \tau_3\ \texttt{list}$
  - ★ The pattern matching requires that also the pattern `[]` and the matched expression `xs` have the same type τ_3 `list` – which holds.

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$ where τ_5 represents whatever will be the type of the `match...` expression.
- Now we add type constraints:
  - ★ The result type τ_5 must be the same type as the result `ys` in case C 1: $\tau_5 = \tau_2$
  - ★ τ_5 must also be the same type as the result in case C 2: $\texttt{x::(xtail @ ys)} : \tau_5$
  - ★ For builtin `::` we have $\texttt{(::)} : \alpha \to \alpha\ \texttt{list} \to \alpha\ \texttt{list}$ where α can be any type.
    - ▶ We need to instantiate $\alpha = \tau_3$ here since $x : \tau_3$.
  - ★ Thus $\tau_5 = \tau_3\ \texttt{list}$ and $\texttt{xtail @ ys} : \tau_3\ \texttt{list}$
  - ★ Thus $\texttt{xtail} : \tau_1$ because `xtail` is used as first argument of `@`.
  - ★ For the pattern `x::xtail` to be type correct, we must thus hold: $\tau_1 = \tau_3\ \texttt{list}$
  - ★ The pattern matching requires that also the pattern `[]` and the matched expression `xs` have the same type τ_3 `list` – which holds.
  - ★ Thus: $\texttt{(@)} : \tau_3\ \texttt{list} \to \tau_3\ \texttt{list} \to \tau_3\ \texttt{list}$

Polymorphic type inference – informally

Given a declaration, for example,

```
let rec (@) xs ys =
   match xs with
   | []       -> ys                    (* C 1 *)
   | x::xtail -> x::(xtail @ ys);;     (* C 2 *)
```

- At the beginning of type inference, we give each identifier a new type variable.
  - ★ This is like saying: I do not know the type of `xs` yet, so it is some type τ_1.

$$\texttt{xs} : \tau_1 \quad \texttt{ys} : \tau_2 \quad \texttt{x} : \tau_3 \quad \texttt{xtail} : \tau_4$$

- For the function being defined here, we can say: $\texttt{(@)} : \tau_1 \to \tau_2 \to \tau_5$ where τ_5 represents whatever will be the type of the `match...` expression.
- Now we add type constraints:
  - ★ The result type τ_5 must be the same type as the result `ys` in case C 1: $\tau_5 = \tau_2$
  - ★ τ_5 must also be the same type as the result in case C 2: $\texttt{x::(xtail @ ys)} : \tau_5$
  - ★ For builtin `::` we have $\texttt{(::)} : \alpha \to \alpha\ \texttt{list} \to \alpha\ \texttt{list}$ where α can be any type.
    - ▶ We need to instantiate $\alpha = \tau_3$ here since $x : \tau_3$.
  - ★ Thus $\tau_5 = \tau_3\ \texttt{list}$ and $\texttt{xtail @ ys} : \tau_3\ \texttt{list}$
  - ★ Thus $\texttt{xtail} : \tau_1$ because `xtail` is used as first argument of `@`.
  - ★ For the pattern `x::xtail` to be type correct, we must thus hold: $\tau_1 = \tau_3\ \texttt{list}$
  - ★ The pattern matching requires that also the pattern `[]` and the matched expression `xs` have the same type τ_3 `list` – which holds.
  - ★ Thus: $\texttt{(@)} : \tau_3\ \texttt{list} \to \tau_3\ \texttt{list} \to \tau_3\ \texttt{list}$
- The most general type or principle type of `@` is thus (renaming τ_3 to α):

$$\texttt{(@)} : \alpha\ \texttt{list} \to \alpha\ \texttt{list} \to \alpha\ \texttt{list}$$

Overloaded Operators and Type inference

A squaring function on integers:

Declaration	Type	
`let square x = x * x`	`int -> int`	Default

Overloaded Operators and Type inference

A squaring function on integers:

Declaration	Type	
`let square x = x * x`	`int -> int`	Default

A squaring function on floats: `square:  float -> float`

Declaration	

Overloaded Operators and Type inference

A squaring function on integers:

Declaration	Type	
`let square x = x * x`	`int -> int`	Default

A squaring function on floats: `square: float -> float`

Declaration	
`let square(x:float) = x * x`	Type the argument

Overloaded Operators and Type inference

A squaring function on integers:

Declaration	Type	
`let square x = x * x`	`int -> int`	Default

A squaring function on floats: `square: float -> float`

Declaration	
`let square(x:float) = x * x`	Type the argument
`let square x:float = x * x`	Type the result

Overloaded Operators and Type inference

A squaring function on integers:

Declaration	Type	
`let square x = x * x`	`int -> int`	Default

A squaring function on floats: `square: float -> float`

Declaration	
`let square(x:float) = x * x`	Type the argument
`let square x:float = x * x`	Type the result
`let square x = x * x: float`	Type expression for the result

Overloaded Operators and Type inference

A squaring function on integers:

Declaration	Type	
`let square x = x * x`	`int -> int`	Default

A squaring function on floats: `square:  float -> float`

Declaration	
`let square(x:float) = x * x`	Type the argument
`let square x:float = x * x`	Type the result
`let square x = x * x:  float`	Type expression for the result
`let square x = x:float * x`	Type a variable

Overloaded Operators and Type inference

A squaring function on integers:

Declaration	Type	
`let square x = x * x`	`int -> int`	Default

A squaring function on floats: `square:  float -> float`

Declaration	
`let square(x:float) = x * x`	Type the argument
`let square x:float = x * x`	Type the result
`let square x = x * x:  float`	Type expression for the result
`let square x = x:float * x`	Type a variable

You can mix these possibilities

Basic types: equality and comparison

Equality and comparison are defined for the basic types of F#, including integers, floats, booleans, characters and strings.

Examples:

```
true < false;;
val it : bool = false

'a' < 'A';;
val it : bool = false

"a" < "ab";;
val it : bool = true
```

Composite Types: equality and comparison

Equality and comparison carry over to composite types
as long as function types are not involved:

Composite Types: equality and comparison

Equality and comparison carry over to composite types
as long as function types are not involved:

Equality is defined structurally on values with the same type:

```
[[1;2]; [3;4;5]] = [[1..2]; [3..5]];;
val it : bool = true
```

Composite Types: equality and comparison

Equality and comparison carry over to composite types
as long as function types are not involved:

Equality is defined structurally on values with the same type:

```
[[1;2]; [3;4;5]] = [[1..2]; [3..5]];;
val it : bool = true
```

Comparison is typically defined using lexicographical ordering:

```
[1; 2; 3] < [1; 4];;
val it : bool = true

(2, [1; 2; 3]) > (2, [1;4]);;
val it : bool = false
```

Tuples

An ordered collection of n values $(v_1, v_2, \ldots, v_n)$ is called an n-tuple

Examples

`(3, false);` *`val it = (3, false) : int * bool`*	2-tuples (pairs)
`(1, 2, ("ab",true));` *`val it = (1, 2, ("ab", true)) : ?`*	3-tuples (triples)

Tuples

An ordered collection of n values $(v_1, v_2, \ldots, v_n)$ is called an n-tuple

Examples

`(3, false);` *`val it = (3, false) : int * bool`*	2-tuples (pairs)
`(1, 2, ("ab",true));` *`val it = (1, 2, ("ab", true)) : ?`*	3-tuples (triples)

Equality defined componentwise, ordering lexicographically

```
(1, 2.0, true) = (2-1, 2.0*1.0, 1<2);;
val it = true :  bool
```

provided $=$ is defined on components

Tuple patterns

Extract components of tuples

```
let ((x,_),(_,y,_)) = ((1,true),("a","b",false));;
val x : int = 1
val y : string = "b"
```

Pattern matching yields bindings

Tuple patterns

Extract components of tuples

```
let ((x,_),(_,y,_)) = ((1,true),("a","b",false));;
val x : int = 1
val y : string = "b"
```

Pattern matching yields bindings

Restriction

```
let (x,x) = (1,1);;
...
 ... ERROR ... 'x' is bound twice in this pattern
```

Tuple patterns

Extract components of tuples

```
let ((x,_),(_,y,_)) = ((1,true),("a","b",false));;
val x : int = 1
val y : string = "b"
```

Pattern matching yields bindings

Restriction

```
let (x,x) = (1,1);;
...
 ... ERROR ... 'x' is bound twice in this pattern
```

Restriction can be circumvented using `when` clauses, for example:

```
let f = function
        | (x,y) when x=y -> x
        | (x,y)          -> x+y
```

Polymorphic types: equality and comparison constraints (I)

Polymorphic types may be accompanied with equality and comparison constraints like:

- when 'a : comparison
- when 'b : equality

For example, there is a built-in function:

$$\texttt{compare}\ x\ y \quad = \quad \begin{cases} > 0 & \text{if } x > y \\ 0 & \text{if } x = y \\ < 0 & \text{if } x < y \end{cases}$$

with the type:

```
'a -> 'a -> int  when 'a : comparison
```

Polymorphic types: equality and comparison constraints (I)

Polymorphic types may be accompanied with equality and comparison constraints like:

- when 'a : comparison
- when 'b : equality

For example, there is a built-in function:

$$\texttt{compare}\ x\ y \quad = \quad \begin{cases} > 0 & \text{if } x > y \\ 0 & \text{if } x = y \\ < 0 & \text{if } x < y \end{cases}$$

with the type:

```
'a -> 'a -> int  when 'a : comparison
```

For example:

```
compare (2, [1; 2; 3]) (2, [1;4]);;
val it : int = -1
```

Polymorphic types: equality and comparison constraints (II)

The built-in function List.contains can be declared as follows:

```
let rec contains x =
   function
   | []    -> false
   | y::ys -> x=y || contains x ys
contains: 'a -> 'a list -> bool when 'a : equality

contains [3;4] [[1..2]; [3..5]];;
val it : bool = false
```

Polymorphic types: equality and comparison constraints (II)

The built-in function List.contains can be declared as follows:

```
let rec contains x =
   function
   | []    -> false
   | y::ys -> x=y || contains x ys
contains: 'a -> 'a list -> bool when 'a : equality

contains [3;4] [[1..2]; [3..5]];;
val it : bool = false
```

Notice:

- The equality constraint in the type
- Lazy (short-circuit) evaluation of $e_1||e_2$ causes termination as soon as an element y equal to x is found
- Yet a recursion following the structure of lists

On let-expressions and lists

Let-expressions

A let-expression e_l has the (verbose) form

```
let x = e1 in e2
```

Let-expressions

A let-expression e_l has the (verbose) form

```
let x = e1 in e2
```

or the following short form exploiting indentation:

```
let x = e1
e2
```

The expression provides a local definition for x in e2.

Let-expressions

A let-expression e_l has the (verbose) form

```
let x = e1 in e2
```

or the following short form exploiting indentation:

```
let x = e1
e2
```

The expression provides a local definition for x in e2.

A let-expression e_l is evaluated in an environment *env* as follows:

Let-expressions

A let-expression e_l has the (verbose) form

```
let x = e1 in e2
```

or the following short form exploiting indentation:

```
let x = e1
e2
```

The expression provides a local definition for x in e2.

A let-expression e_l is evaluated in an environment *env* as follows:

If

1. v_1 is the value obtained by evaluating e_1 in *env*,

Let-expressions

A let-expression e_l has the (verbose) form

```
let x = e1 in e2
```

or the following short form exploiting indentation:

```
let x = e1
e2
```

The expression provides a local definition for x in e2.

A let-expression e_l is evaluated in an environment *env* as follows:

If

1. v_1 is the value obtained by evaluating e_1 in *env*,
2. env' is obtained by adding binding $x \mapsto v_1$ to *env* and

Let-expressions

A let-expression e_l has the (verbose) form

```
let x = e1 in e2
```

or the following short form exploiting indentation:

```
let x = e1
e2
```

The expression provides a local definition for x in e2.

A let-expression e_l is evaluated in an environment *env* as follows:

If

1. v_1 is the value obtained by evaluating e_1 in *env*,
2. env' is obtained by adding binding $x \mapsto v_1$ to *env* and
3. v_2 is the value obtained by evaluating e_2 in env'

Let-expressions

A let-expression e_l has the (verbose) form

```
let x = e1 in e2
```

or the following short form exploiting indentation:

```
let x = e1
e2
```

The expression provides a local definition for x in e2.

A let-expression e_l is evaluated in an environment *env* as follows:

If

1. v_1 is the value obtained by evaluating e_1 in *env*,
2. env' is obtained by adding binding $x \mapsto v_1$ to *env* and
3. v_2 is the value obtained by evaluating e_2 in env'

then

$$(\texttt{let}\ x = e_1\ \texttt{in}\ e_2, env) \rightsquigarrow (v_2, env)$$

Let-expression – an example

```
let g y = let a = 6
          let b = y + a
          y + b;;
val g : int -> int

g 1;;
val it : int = 8
```

Note: a and b are not visible outside of g

Let-expression – an example

```
let g y = let a = 6
          let b = y + a
          y + b;;
val g : int -> int

g 1;;
val it : int = 8
```

Note: a and b are not visible outside of g

Evaluation ?

Pattern matching on results of recursive calls

```
sumProd [x0;x1; ...;xn-1]
              =   ( x0 + x1 + ... + xn-1 , x0 * x1 * ... * xn-1 )
sumProd []    =   (0,1)
```

Pattern matching on results of recursive calls

```
sumProd [x0;x1; ...;xn-1]
              =   ( x0 + x1 + ... + xn-1 , x0 * x1 * ... * xn-1 )
sumProd []    =   (0,1)
```

The declaration is based on the recursion formula:

$$\texttt{sumProd [}x_0\texttt{;}x_1\texttt{; ...;}x_{n-1}\texttt{]} = \texttt{(}x_0\texttt{ + rSum,}x_0\texttt{ * rProd)}$$

where $\texttt{(rSum,rProd)} = \texttt{sumProd [}x_1\texttt{; ...;}x_{n-1}\texttt{]}$

Pattern matching on results of recursive calls

$$\begin{aligned}
\texttt{sumProd}\ [x_0;x_1;\ldots;x_{n-1}] &= (\ x_0 + x_1 + \ldots + x_{n-1}\ ,\ x_0 * x_1 * \ldots * x_{n-1}\ ) \\
\texttt{sumProd}\ [] &= (0,1)
\end{aligned}$$

The declaration is based on the recursion formula:

$$\texttt{sumProd}\ [x_0;x_1;\ldots;x_{n-1}] = (x_0 + \texttt{rSum}, x_0 * \texttt{rProd})$$

where $(\texttt{rSum},\texttt{rProd}) = \texttt{sumProd}\ [x_1;\ldots;x_{n-1}]$

This gives the declaration:

```
let rec sumProd =
   function
   | []      -> (0,1)
   | x::rest -> let (rSum,rProd) = sumProd rest
                in (x+rSum,x*rProd);;
val sumProd : int list -> int * int

sumProd [2;5];;
val it : int * int = (7, 10)
```

A blackboard exercise

A function from the `List` library:

- `List.unzip([(`x_0, y_0`);(`x_1, y_1`);...;(`x_{n-1}, y_{n-1}`)]`
  $=$ `([`x_0`;`x_1`;...;`x_{n-1}`],[`y_0`;`y_1`;...;`y_{n-1}`])`

The Syntax unlocked so far:

Expressions

An expression can be any of the following:

- A constant
- An identifier
- If e, e_1, $\ldots e_n$ are expressions and $\oplus$ is an infix operator (like $+$, $*$, $<$, $=$,...), $p_1, \ldots, p_n$ are patterns, cs is a set of pattern matching clauses, and d is a declaration, then also the following are expressions:
  - ★ $e_1 \oplus e_2$
  - ★ `if` e `then` e_1 `else` e_2
  - ★ (e)
  - ★ $e_1\ e_2$
  - ★ $(e_1, \ldots, e_n)$
  - ★ `match` e `with` cs
  - ★ `fun` $\mathtt{p}_1\ \mathtt{p}_2\ \ldots\ \mathtt{p}_n \rightarrow \mathtt{e}$
  - ★ `function` cs
  - ★ d `in` e
  - ★ []
  - ★ $e_1 :: e_2$

The Syntax unlocked so far:

Patterns

A pattern can be any of the following:

- A constant
- An identifier
- The wildcard _
- If $p_1, \ldots p_n$ are patterns and e and expression, then also the following are patterns:
  - ★ $(p_1, \ldots, p_n)$
  - ★ $p_1 \mid p_2$
  - ★ `[]`
  - ★ $\texttt{p}_1\texttt{::p}_2$
  - ★ $\texttt{p}_1$ `when e`

Pattern matching clauses

If $p_1, \ldots, p_n$ are patterns and $e_1, \ldots, e_n$ are expressions then the following forms a set of pattern matching clauses:

$$\begin{aligned} \mid p_1 &\rightarrow e_1 \\ &\ldots \\ \mid p_n &\rightarrow e_n \end{aligned}$$

The Syntax unlocked so far:

Declarations

If e is an expression and f is an identifier, and $p_1, \ldots, p_n$ $(n \geq 0)$ are patterns then the following are declarations:

- `let f` $\mathtt{p_1}$ `...` $\mathtt{p_n}$ `= e`
- `let rec f` $\mathtt{p_1}$ `...` $\mathtt{p_n}$ `= e`

What is the value of *a*?

Consider

```
let pi = 3.14;;

let ca r = pi * r * r;;

let a = let pi = 1.0;;
        ca 1.0;;

 a?
```

Semantics of a function: A closure

Consider a declaration of `f` in an environment $env = [a \mapsto 4, b \mapsto true]$:

```
let f x = x+a
```

Semantics of a function: A closure

Consider a declaration of `f` in an environment $env = [a \mapsto 4, b \mapsto true]$:

```
let f x = x+a
```

The resulting environment is:

$$[a \mapsto 4, b \mapsto true, f \mapsto cl_f]$$

Semantics of a function: A closure

Consider a declaration of `f` in an environment $env = [a \mapsto 4, b \mapsto true]$:

```
let f x = x+a
```

The resulting environment is:

$$[a \mapsto 4, b \mapsto true, f \mapsto cl_f]$$

where the *value of f* is a closure $cl_f =$

$$([x], x + a, [a \mapsto 4])$$

consisting of

- the argument list: [x]
- the body of f: x+a
- the environment with bindings for the *free* variables: $[a \mapsto 4]$

Semantics of a function: A closure

Consider a declaration of `f` in an environment $env = [a \mapsto 4, b \mapsto true]$:

```
let f x = x+a
```

The resulting environment is:

$$[a \mapsto 4, b \mapsto true, f \mapsto cl_f]$$

where the *value of f* is a closure $cl_f =$

$$([x], x + a, [a \mapsto 4])$$

consisting of

- the argument list: [x]
- the body of f: x+a
- the environment with bindings for the *free* variables: $[a \mapsto 4]$

The bindings in a closure's environment are determined at the place where the function is declared static binding

Function application

Let *env* be an environment, where the closure for f is

$$cl_f = ([x], e, env_f)$$

Function application

Let env be an environment, where the closure for f is

$$cl_f = ([x], e, env_f)$$

The application $f(a)$ is evaluated as follows in env:

$$\begin{array}{ll} & (f(a), env) \\ \rightsquigarrow & (e, [x \mapsto v] + env_f) \end{array}$$

Function application

Let env be an environment, where the closure for f is

$$cl_f = ([x], e, env_f)$$

The application $f(a)$ is evaluated as follows in env:

$$\begin{array}{rl} & (f(a), env) \\ \rightsquigarrow & (e, [x \mapsto v] + env_f) \end{array}$$

where

- v is the value of a in env

Function application

Let env be an environment, where the closure for f is

$$cl_f = ([x], e, env_f)$$

The application $f(a)$ is evaluated as follows in env:

$$\begin{array}{rl} & (f(a), env) \\ \rightsquigarrow & (e, [x \mapsto v] + env_f) \end{array}$$

where

- v is the value of a in env
- $[x \mapsto v] + env_f$ is the environment obtained by adding the binding from $x \mapsto v$ to env_f.

Static binding: an example

Consider

```
let pi = 3.14;;
```

env1 = ?

```
let ca r = pi * r * r;;
```

env2 = ?

```
let a = let pi = 1.0
```

env3 = ?

```
        ca 1.0;;
```

env4 = ?

a?

Exercises

- Some programming exercises: HR 4.3, 4.8, 4.9, 4.12
- Some exercises that are supposed to be done on paper: HR 4.16, 4.17

  ★ Derive the type of the given functions like we have done in the lecture.

  Of course, the F# interpreter could also compute the type automatically, but the task is to give the inference steps like in the lecture.

  ★ Try to understand how each function works and how the given expression evaluates.

  Also here one could use the F# interpreter to compute it for some concrete input lists, but the task is to obtain a short description of the evaluation for generic lists like $[x_0; \ldots; x_{n-1}]$.